

Documentation for ALPS_Visio_AddIn_rewrite

This Visio AddIn adds an Importer for [Abstract Layered PASS \(ALPS\)](#) diagrams.

Abstract Layered PASS

TODO

API

Code

We will discuss the code in the order of execution. Filenames are relative to the project's root directory.

[ThisAddIn.Designer.cs](#)

This file holds the Visio AddIn base. Further functionality gets added by [ThisAddIn](#).

[ThisAddIn.cs](#)

```
public partial class ThisAddIn
{
    private void InternalStartup()
    {
        this.Startup += new System.EventHandler(ThisAddIn_Startup);
    }
}
```

When the AddIn is loaded we create our [Ribbon](#).

```
private void ThisAddIn_Startup(object sender, System.EventArgs e)
{
    new ALPSRibbon();
}
```

Currently there is also some logic for retrieving the active document and the AddIn instance.

```
Application.WindowActivated += Application_WindowActivated;
Application.DocumentOpened += Application_DocumentOpened;
Application.DocumentCreated += Application_DocumentCreated;
this.setActiveDocument();

currentInstance = this;
}

private Visio.Document activeDoc;
private void setActiveDocument() {this.activeDoc =
```

```

Application.ActiveDocument;}
    private void Application_WindowActivated(Window window)
{this.setActiveDocument();}
    private void Application_DocumentOpened(IVDocument doc)
{this.setActiveDocument();}
    private void Application_DocumentCreated(IVDocument doc)
{this.setActiveDocument();}

    private static ThisAddIn currentInstance;
    public static ThisAddIn getInstance() {return currentInstance;}

```

This comes from the original version passing the active document to the [OWLImporter](#), even though it did not use it.

```

    private static string showOWLFileDialog()
    {
        OpenFileDialog openFileDialog1 = new OpenFileDialog();
        openFileDialog1.Filter = "Ontology Files (.owl)|*.owl| RDF Files
(*.rdf)|*.rdf";
        openFileDialog1.ShowDialog();
        return openFileDialog1.FileName;
    }

    public void loadOWLFile()
    {
        string fileName = showOWLFileDialog();
        if (fileName == "") return;
        OWLImporter importer = new OWLImporter(fileName);
        importer.parse(null, activeDoc);

        VisioHelper.openStencil(VisioHelper.VisioStencils.SID_STENCIL); // why
here
    }
}

```

VisioHelper.cs

This file is a collection of helper functions for the communication with Visio.

```

public static class VisioHelper
{

```

This function enables and disables the VBA Macro.

```

    public static void setVBAListenersRunning(Boolean newStatus)
    {

```

```
//...
}
```

An enum stores the stencils.

```
public enum VisioStencils
{
    SID_STENCIL,
    SBD_STENCIL
}

public static Visio.Document openStencil(VisioStencils stencil)
{
    Visio.Documents visioDocs = Globals.ThisAddIn.Application.Documents;
    try
    {
        switch (stencil)
        {
```

The stencil file gets loaded and returned. For some reason this is outsourced to [ShapeFinder](#).

```
        case VisioStencils.SID_STENCIL:
            Visio.Document sidShapes =
visioDocs.OpenEx(ShapeFinder.getSIDName(),
(short)Visio.VisOpenSaveArgs.visOpenDocked);
            return sidShapes;
        case VisioStencils.SBD_STENCIL:
            Visio.Document sbdShapes =
visioDocs.OpenEx(ShapeFinder.getSBDName(),
(short)Visio.VisOpenSaveArgs.visOpenDocked);
            return sbdShapes;
        }
    }
    catch (System.Runtime.InteropServices.COMException e)
    {
```

If the file is not found, an error message is thrown and this function returns `null`.

```
//...
}
return null;
}
```

An enum stores the shape types.

```
public enum ShapeType
{
    SBD, SID
}
```

This function drops a shape onto the page. For some reason it expects points and an original element, which never get used.

```
public static Visio.Shape place(ShapeType shapeType, Visio.Page page, string
masterType, IList<ISimple2DVisualizationPoint> points = null,
IPASSProcessModelElement originalElement = null)
{
    Visio.Document stencil = null;
    switch (shapeType)
    {
        case ShapeType.SBD:
            stencil = VisioHelper.openStencil(VisioStencils.SBD_STENCIL);
            break;
        case ShapeType.SID:
            stencil = VisioHelper.openStencil(VisioStencils.SID_STENCIL);
            break;
    }
    Visio.Master sidMaster = stencil.Masters.get_ItemU(masterType);

    Visio.Shape droppedShape = page.Drop(sidMaster, 0, 0);
    return droppedShape;
}
```

This function creates a new SID page.

```
public static Visio.Page CreateSIDPage(string name, string nameU, string
modelURI, string extends, string implements, string priority)
{
```

For some reason the document count gets checked, but there should always exist a document at this point.

```
Visio.Application addin = Globals.ThisAddIn.Application;
if (addin.Documents.Count < 1)
{
    addin.Documents.Add("");
}

Visio.Page page =
Globals.ThisAddIn.Application.ActiveDocument.Pages.Add();
page.Name = name;
page.NameU = nameU;
```

The SID page properties are set up in the page sheet. For some reason `nameU` is set as the page layer.

```
//...  
  
return page;  
}
```

This function creates a new SBD page, belonging to a subject on a SID page.

```
public static Visio.Page CreateSBDPage(Visio.Page sidPage, string name, string  
nameU, Visio.Shape subjectShape)  
{  
    Visio.Page page =  
Globals.ThisAddIn.Application.ActiveDocument.Pages.Add();  
    page.Name = name;  
    page.NameU = nameU;
```

The SBD page properties are set up in the page sheet.

```
//...
```

Also, the hyperlinks between the subject and the SBD are set up.

```
page.PageSheet.AddNamedRow((short)Visio.VisSectionIndices.visSectionHyperlink,  
ALPSConstants.alpsHyperlinksLinkedSIDPage, 0);  
  
page.PageSheet.Hyperlinks.ItemU[ALPSConstants.alpsHyperlinksLinkedSIDPage].SubAddress  
= sidPage.NameU;  
  
subjectShape.Hyperlinks.ItemU[ALPSConstants.alpsHyperlinkTypeLinkedSBD].SubAddress  
= "" + page.NameU + "";  
  
return page;  
}
```

ALPSConstants.cs

This class just defines constants used throughout the code.

```
public static class ALPSConstants
{
    //...
}
```

ALPSGlobalFunctions.cs

This class just defines functions used throughout the code. For some reason this is separate from the constants.

```
public class ALPSGlobalFunctions
{
    //...
}
```

ShapeFinder.cs

This class implements functions for finding the newest shapes and returning their file names.

```
public static class ShapeFinder
{
    //...
}
```

menu/ALPSRibbonDesigner.cs

```
partial class ThisRibbonCollection
{
    internal ALPSRibbon ALPSRibbon
    {
        get { return this.GetRibbon<ALPSRibbon>(); }
    }
}

partial class ALPSRibbon : Microsoft.Office.Tools.Ribbon.RibbonBase
{
    private System.ComponentModel.IContainer components = null;

    public ALPSRibbon() : base(Globals.Factory.GetRibbonFactory())
    {
        InitializeComponent();
    }

    protected override void Dispose(bool disposing)
    {
        if (disposing && (components != null))
        {
            components.Dispose();
        }
    }
}
```

```

    {
        components.Dispose();
    }
    base.Dispose(disposing);
}

```

The ribbon consists of one tab with one group with one button.

```

internal Microsoft.Office.Tools.Ribbon.RibbonTab tab1;
internal Microsoft.Office.Tools.Ribbon.RibbonGroup group1;
internal Microsoft.Office.Tools.Ribbon.RibbonButton loadOWLFile;

private void InitializeComponent()
{
    this.tab1 = this.Factory.CreateRibbonTab();
    this.tab1.SuspendLayout();
    this.group1 = this.Factory.CreateRibbonGroup();
    this.group1.SuspendLayout();
    this.loadOWLFile = this.Factory.CreateRibbonButton();
    this.SuspendLayout();

    this.tab1.Label = "ALPS/PASS ADDIN";
    this.tab1.Groups.Add(this.group1);

    this.group1.Label = "OWL PASS Tools";
    this.group1.Items.Add(this.loadOWLFile);

    this.loadOWLFile.Name = "loadOWLFile";
    this.loadOWLFile.Label = "Import OWL";
    this.loadOWLFile.SuperTip = "Use this tool to import PASS and ALPS Process Models from OWL Files based on the standard pass ontology";
    this.loadOWLFile.Image =
global::ALPS_Visio_AddIn_rewrite.Properties.Resources.owlIcon2;
    this.loadOWLFile.ShowImage = true;
    this.loadOWLFile.ControlSize =
Microsoft.Office.Core.RibbonControlSize.RibbonControlSizeLarge;
    this.loadOWLFile.Click += new
Microsoft.Office.Tools.Ribbon.RibbonControlEventHandler(this.loadOWLFile_Click);

    this.RibbonType = "Microsoft.Visio.Drawing";
    this.Tabs.Add(this.tab1);

    this.tab1.ResumeLayout(true);
    this.group1.ResumeLayout(true);
    this.ResumeLayout(true);
}
}

```

For some reason the ClickEvent is outsourced to another file.

```
public partial class ALPSRibbon
{
    private void loadOWLFile_Click(object sender, RibbonControlEventArgs e)
    {
```

This calls the function in [ThisAddIn](#).

```
        Globals.ThisAddIn.loadOWLFile();
    }
}
```

OWL/OWLImporter.cs

```
public class OWLImporter
{
```

The importer parses using the [ALPS API](#).

```
private IPASSReaderWriter parser = PASSReaderWriter.GetInstance();
```

For some reason also the importer is static for each file.

```
private string fileName;

public OWLImporter(string fileName)
{
```

This enables type checking for reflection and sets the [class factory](#).

```
ReflectiveEnumerator.addAssemblyToCheckForTypes(Assembly.GetExecutingAssembly());
parser.setModelElementFactory(new VisioClassFactory());
```

The parsing structure is defined by the PASS and [ALPS](#) standards.

```
parser.loadOWLParsingStructure(new List<string>
{
```



```

        "../../Resources/standard_PASS_ont_v_1.1.0.owl",
        "../../Resources/ALPS_ont_v_0.8.0.owl"
    });

    this.fileName = fileName;
}

```

For some reason the parser expects the active document as its argument but never uses it.

```

public void parse(Visio.Page mainPage, Visio.Document activeDoc)
{

```

The parser loads all the models from the file.

```

        IList<IPASSProcessModel> passProcessModels = parser.loadModels(new
List<string> { fileName });

        VisioHelper.openStencil(VisioHelper.VisioStencils.SID_STENCIL); // why
here

```

The VBA Macro can disrupt the exporting process, thus it gets temporarily disabled.

```

VisioHelper.setVBAListenersRunning(false);

```

For some reason only the first model gets exported to Visio.

```

        if (passProcessModels.Count > 0 && passProcessModels[0] is
IVisioExportable exportable)
        {

```

Exporting utilizes the [IVisioExportable](#) interface.

```

            exportable.exportToVisio(mainPage);
        }
        VisioHelper.setVBAListenersRunning(true);
    }
}

```

OWLShapes/VisioClassFactory.cs

```
public class VisioClassFactory : BasicPASSProcessModelElementFactory
{
```

The class factory tries to find the correct class for an element.

```
    protected override KeyValuePair<IParseablePASSProcessModelElement, string>
decideForElement(IDictionary<IParseablePASSProcessModelElement, string>
possibleElements)
    {
        foreach (KeyValuePair<IParseablePASSProcessModelElement, string> pair in
possibleElements)
        {
```

The classes are generalized by the [IVisioExportable](#) interface.

```
            if (pair.Key is IVisioExportable) return pair;
        }
```

If no class exists, the default class from the [ALPS API](#) is used.

```
        return base.decideForElement(possibleElements);
    }
}
```

OWLShapes/IVisioExportable.cs

```
public interface IVisioExportable
{
```

This is the general method for exporting elements to Visio. For some reason the bounds parameter is expected, but not used in any implementation.

```
    void exportToVisio(Visio.Page currentPage, ISimple2DVisualizationBounds bounds
= null);
}
```

OWLShapes/IVisioExportableWithShape.cs

```
public interface IVisioExportableWithShape : IVisioExportable
{
```

This makes the methods of the [IShapeExport](#) accessible.

```
Visio.Shape getShape();
void setShape(Visio.Shape shape);
```

Elements with Shapes need their visual information to be set up.

```
bool prep2DInfo();
}
```

OWLShapes/ExportFunctionality/IShapeExport.cs

```
public interface IShapeExport
{
```

This method should export the export's element to Visio and place its shape using [VisioHelper](#). As in its place method, the original element never gets used. The points are used in [StateExport](#) and [SubejctExport](#).

```
void export(ShapeType shapeType, Visio.Page page, string masterType,
IList<ISimple2DVisualizationPoint> points = null, IPASSProcessModelElement
originalElement = null);
```

These methods get and set the shape of the element.

```
Visio.Shape getShape();
void setShape(Visio.Shape shape);
}
```

OWLShapes/ExportFunctionality/PASSProcessModelExport.cs

This is the specialization for [IShapeExport](#) for elements of PASS process models.

```
public class PASSProcessModelElementExport : IShapeExport
{
```

The element to be exported is passed to the constructor.

```
readonly IPASSProcessModelElement element;
public PASSProcessModelElementExport(IPASSProcessModelElement element)
{
    this.element = element;
}
```

The element's shape is stored and accessible as given by [IShapeExport](#).

```
protected Visio.Shape shape;
public Visio.Shape getShape()
{
    return shape;
}
public void setShape(Visio.Shape shape)
{
    this.shape = shape;
}

public virtual void export(ShapeType shapeType, Visio.Page page, string
masterType, IList<ISimple2DVisualizationPoint> points = null,
IPASSProcessModelElement originalModelElement = null)
{
```

The shape gets placed on the page.

```
shape = place(shapeType, page, masterType, points, originalModelElement);
```

The shape's properties and labels are set up.

```
shape.CellsU["Prop." +
ALPSConstants.alpsPropertieTypeModelComponentID].Formula = "\"" +
element.getModelComponentID() + "\"";
if (element.getComments().Count > 0) shape.CellsU["Prop." +
ALPSConstants.alpsPropertieTypeComment].Formula = "\"" + string.Join(";",
element.getComments()) + "\"";
shape.CellsU["Prop." +
ALPSConstants.alpsPropertieTypeModelComponentType].Formula = "\"" +
element.GetType() + "\"";

string englishLabel = getEnglishLabel(element.getModelComponentLabels(),
out IList<IStringWithExtra> otherLabels);
if (englishLabel == null)
{
    englishLabel = otherLabels.FirstOrDefault()?.getContent();
```

```

        if (otherLabels.Count > 0) otherLabels.RemoveAt(0);
    }
    shape.CellsU["Prop." + ALPSConstants.alpsPropertieTypeLabel].Formula =
"\\"" + englishLabel + "\"";
    foreach (IStringWithExtra otherLabel in otherLabels)
    {
        string newRowName = "label" + otherLabel.getExtra().ToUpper();
        shape.AddNamedRow((short)Visio.VisSectionIndices.visSectionProp,
newRowName, (short)Visio.VisRowTags.visTagDefault);
        shape.CellsU["Prop." + newRowName].Formula = "\"" +
otherLabel.getContent() + "\"";
    }
}

```

This is a helper method to filter the english label from the rest. For some reason it is here instead of in [VisioHelper](#) with the other helper methods.

```

protected string getEnglishLabel(IList<IStringWithExtra> allLabels, out
IList<IStringWithExtra> nonEnglishLabels)
{
    //...
}

```

Export

For some reason every class implements a constructor that just calls the base class constructor.

Every class will return a new object of itself as the parsed instance.

```

protected Class() { }
public override IParseablePASSProcessModelElement getParsedInstance()
{
    return new Class();
}

```

OWLShapes/VisioPASSProcessModel.cs

```

public class VisioPASSProcessModel : PASSProcessModel, IVisioExportable
{
    public void exportToVisio(Visio.Page currentPage)
    {

```

Each [ModelLayer](#) creates a page and gets exported onto it. For some reason, the page gets created before the exporting condition is checked.

```

        foreach (IModellLayer modellLayer in
getAllElements().Values.OfType<IModellLayer>())
    {

```

The SID page name is the model layer's ID. For some reason `nameU` is not set.

```

        Visio.Page page =
VisioHelper.CreateSIDPage(modellLayer.getModelComponentID(), " ",
modellLayer.getUriModelComponentID(), " ", " ", " ");

        if (modellLayer is IVisioExportable exportable)
exportable.exportToVisio(page);
    }
}

```

OWLShapes/VisioModellLayer.cs

```

public class VisioModellLayer : ModellLayer, IVisioExportable
{
    public void exportToVisio(Visio.Page currentPage)
    {

```

For some reason the page's bounds are set before any element is placed onto it.

```

        setPageBounds(currentPage);

        foreach (IPASSProcessModelElement modelElement in getElements().Values)
        {
            if (!(modelElement is IVisioExportable exportable)) continue;

```

Elements with Shapes set up their visual information.

```

            if (exportable is IVisioExportableWithShape shapeExportable)
shapeExportable.prep2DInfo();

```

All SID elements ([FullySpecifiedSubject](#), [MessageExchange](#), [MessageExchangeList](#)) are exported to the current page.

```

            if (exportable is ISubject || exportable is IMessageExchange ||
exportable is IMessageExchangeList) exportable.exportToVisio(currentPage);

```

```
    }
}
```

The average size ratio of all elements and the calculated page width and height are set up in the page sheet.

```
private void setPageBounds(Visio.Page currentPage)
{
    //...
}
}
```

OWLShapes/InteractionDescribing/VisioFullySpecifiedSubject.cs

```
public class VisioFullySpecifiedSubject : FullySpecifiedSubject,
IVisioExportableWithShape
{
```

This stores the type of shape to be placed.

```
private const string type = ALPSConstants.alpsSIDMasterStandardActor;
```

The constructors set up an [IShapeExport](#). For some reason the public constructor is also declared the same, but never used.

```
private readonly IShapeExport export;
protected VisioFullySpecifiedSubject()
{
    export = new SubjectExport(this);
}

public void exportToVisio(Visio.Page currentPage)
{
    export.export(VisioHelper.ShapeType.SID, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
```

A new SBD page is created with the [VisioSubjectBehavior](#).

```
Visio.Page currentSBDPage = VisioHelper.CreateSBDPage(currentPage, ("SBD:
" + getModelComponentID()), ("" + getModelComponentID()), this.getShape());
```

```

        if (getSubjectBaseBehavior() is IVisioExportable exportable)
            exportable.exportToVisio(currentSBDPage);
    }

    public bool prep2DInfo()
    {

```

If the 2D information exists, it gets stored in this object.

```

        if (this is IHasSimple2DVisualizationBox bounds)
        {
            //...
            return true;
        }
        else return false;
    }
}

```

OWLShapes/ExportFunctionality/SubjectExport.cs

```

public class SubjectExport : PASSProcessModelElementExport
{

```

The element to be exported is stored as a subject.

```

    readonly ISubject subject;
    public SubjectExport(ISubject subject) : base(subject)
    {
        this.subject = subject;
    }

    public override void export(ShapeType shapeType, Visio.Page page, string
masterType, IList<ISimple2DVisualizationPoint> points = null,
IPASSProcessModelElement originalElement = null)
    {
        base.export(shapeType, page, masterType, points, originalElement);
    }

```

The shape's properties and bounds are set up.

```

        //...
    }
}

```


OWLShapes/VisioSubjectBehavior.cs

```
public class VisioSubjectBehavior : SubjectBehavior, IVisioExportableWithShape
{
    protected VisioSubjectBehavior() {}

    public void exportToVisio(Visio.Page currentPage)
    {
        foreach (IBehaviorDescribingComponent component in
behaviorDescriptionComponents.Values)
        {
            if (!(component is IVisioExportable exportable)) continue;

```

Elements with Shapes set up their visual information.

```
            if (exportable is IVisioExportableWithShape shapeExportable)
shapeExportable.prep2DInfo();

```

All SBD elements (States and Transitions) are exported to the current page.

```
            if (exportable is IState || exportable is ITransition)
exportable.exportToVisio(currentPage);
        }
    }

```

For some reason this class is treated as if it had a shape, even though it does not have any.

```
    public Visio.Shape getShape()
    {
        throw new NotImplementedException();
    }

    public void setShape(Visio.Shape shape)
    {
        throw new NotImplementedException();
    }

```

For some reason also this method defaults to false, even if shapes exist (see later).

```
    public bool prep2DInfo()
    {
        return false;
    }
}

```

OWLShapes/BehaviorDescribing/States/VisioDoState.cs

```
public class VisioDoState : DoState, IVisioExportableWithShape
{
    private const string type = ALPSConstants.alpsSBDMasterDoState;

    private readonly IShapeExport export;
    protected VisioDoState()
    {
        export = new StateExport(this);
    }

    public void exportToVisio(Page currentPage)
    {
        export.export(VisioHelper.ShapeType.SBD, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
    }

    public bool prep2DInfo()
    {
        if (this is IHasSimple2DVisualizationBox bounds)
        {
            //...
            return true;
        }
        else return false;
    }
}
```

OWLShapes/BehaviorDescribing/States/VisioRecieveState.cs

```
public class VisioReceiveState : ReceiveState, IVisioExportableWithShape
{
    private const string type = ALPSConstants.alpsSBDMasterReceiveState;

    private readonly IShapeExport export;
    protected VisioReceiveState()
    {
        export = new StateExport(this);
    }

    public void exportToVisio(Page currentPage)
    {
        export.export(VisioHelper.ShapeType.SBD, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
```

```

    ()), this);
    }

    public bool prep2DInfo()
    {
        if (this is IHasSimple2DVisualizationBox bounds)
        {
            //...
            return true;
        }
        else return false;
    }
}

```

OWLShapes/BehaviorDescribing/States/VisioSendState.cs

```

public class VisioSendState : SendState, IVisioExportableWithShape
{
    private const string type = ALPSConstants.alpsSBDMasterSendState;

    private readonly IShapeExport export;
    protected VisioSendState()
    {
        export = new StateExport(this);
    }

    public void exportToVisio(Page currentPage)
    {
        export.export(VisioHelper.ShapeType.SBD, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
    }

    public bool prep2DInfo()
    {
        if (this is IHasSimple2DVisualizationBox bounds)
        {
            //...
            return true;
        }
        else return false;
    }
}

```

OWLShapes/ExportFunctionality/StateExport.cs

```

public class StateExport : PASSProcessModelElementExport
{

```

```

    readonly IState state;
    public StateExport(IState state) : base(state)
    {
        this.state = state;
    }

    public override void export(ShapeType shapeType, Visio.Page page, string
masterType, IList<ISimple2DVisualizationPoint> points = null,
IPASSProcessModelElement originalElement = null)
    {
        base.export(shapeType, page, masterType, points, originalElement);
    }

```

The shape's properties and bounds are set up.

```

        //...
    }
}

```

OWLShapes/ExportFunctionality/Transitions/VisioDoTransition.cs

```

public class VisioDoTransition : DoTransition, IVisioExportableWithShape
{
    private const string type = ALPSConstants.alpsSBDMasterStandardTransition;

    private readonly IShapeExport export;
    protected VisioDoTransition()
    {
        export = new TransitionExport(this);
    }

    public void exportToVisio(Page currentPage)
    {
        export.export(VisioHelper.ShapeType.SBD, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
    }
}

```

Some properties are set up.

```

        //...
    }
}

```

OWLShapes/ExportFunctionality/Transitions/VisioRecieveTransition.cs

```

public class VisioReceiveTransition : ReceiveTransition, IVisioExportableWithShape
{
    private const string type = ALPSConstants.alpsSBDMasterReceiveTransition;

    private readonly IShapeExport export;
    protected VisioReceiveTransition()
    {
        export = new TransitionExport(this);
    }

    public void exportToVisio(Page currentPage)
    {
        export.export(VisioHelper.ShapeType.SBD, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
    }
}

```

Some properties are set up.

```

        //...
    }
}

```

OWLShapes/ExportFunctionality/Transitions/VisioSendTransition.cs

```

public class VisioSendTransition : SendTransition, IVisioExportableWithShape
{
    private const string type = ALPSConstants.alpsSBDMasterSendTransition;

    private readonly IShapeExport export;
    protected VisioSendTransition()
    {
        export = new TransitionExport(this);
    }

    public void exportToVisio(Page currentPage)
    {
        export.export(VisioHelper.ShapeType.SBD, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
    }
}

```

Some properties are set up.

```

        //...
    }
}

```

```
}
```

OWLShapes/ExportFunctionality/Transitions/VisioTimeTransition.cs

```
public class VisioTimeTransition : TimeTransition, IVisioExportableWithShape
{
    private const string type = ALPSConstants.alpsSBDMasterTimeTransition;

    private readonly IShapeExport export;
    protected VisioTimeTransition()
    {
        export = new TransitionExport(this);
    }

    public void exportToVisio(Page currentPage)
    {
        export.export(VisioHelper.ShapeType.SBD, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
    }
}
```

The time out condition is set up.

```
        //...
    }
}
```

OWLShapes/ExportFunctionality/Transitions/VisioSendingFailedTransition.cs

```
public class VisioSendingFailedTransition : SendingFailedTransition
{
    private const string type =
ALPSConstants.alpsSBDMasterSendingFailedTransition;

    private readonly IShapeExport export;
    protected VisioSendingFailedTransition()
    {
        export = new TransitionExport(this);
    }

    public void exportToVisio(Page currentPage)
    {
        export.export(VisioHelper.ShapeType.SBD, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
    }
}
```

```

    }
}

```

OWLShapes/ExportFunctionality/Transitions/VisioUserCancelTransition.cs

```

public class VisioUserCancelTransition : UserCancelTransition,
IVisioExportableWithShape
{
    private const string type = ALPSConstants.alpsSBDMasterUserCancel;

    private readonly IShapeExport export;
    protected VisioUserCancelTransition()
    {
        export = new TransitionExport(this);
    }

    public void exportToVisio(Page currentPage)
    {
        export.export(VisioHelper.ShapeType.SBD, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
    }
}

```

OWLShapes/ExportFunctionality/TransitionExport.cs

```

public class TransitionExport : PASSProcessModelElementExport
{
    readonly ITransition transition;
    public TransitionExport(ITransition transition) : base(transition)
    {
        this.transition = transition;
    }

    public override void export(ShapeType shapeType, Visio.Page page, string
masterType, IList<ISimple2DVisualizationPoint> points = null,
IPASSProcessModelElement originalElement = null)
    {
        base.export(shapeType, page, masterType, points, originalElement);
    }
}

```

The shape's properties and bounds are set up.

```

        //...
    }
}

```

Work In Progress

The following classes are very unfinished and probably not working as intended.

OWLShapes/InteractionDescribing/VisioMessageExchange.cs

```
public class VisioMessageExchange : MessageExchange, IVisioExportableWithShape
{
    private const string type =
        ALPSConstants.alpsSIDMasterStandardMessageConnector;

    private readonly IShapeExport export;
    protected VisioMessageExchange()
    {
        export = new MessageExchangeExport(this);
    }

    public void exportToVisio(Visio.Page currentPage)
    {
```

For some reason a check exists, if this object's shape already got exported.

```
        if (getShape() != null) return;

        export.export(VisioHelper.ShapeType.SID, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
```

The [VisioMessageSpecification](#) is exported to the current page.

```
        if (getMessageType() is IVisioExportable exportable)
        {
            exportable.exportToVisio(currentPage);
        }
    }
}
```

OWLShapes/ExportFunctionality/MessageExchangeExport.cs

```
public class MessageExchangeExport : PASSProcessModelElementExport
{
    readonly IMessageExchange messageExchange;
    public MessageExchangeExport(IMessageExchange messageExchange) :
base(messageExchange)
```



```

    {
        this.messageExchange = messageExchange;
    }

    public override void export(ShapeType shapeType, Visio.Page page, string
masterType, IList<ISimple2DVisualizationPoint> points = null,
IPASSProcessModelElement originalElement = null)
    {
        base.export(shapeType, page, masterType, points, originalElement);
    }

```

The dimensions are set up.

```

        if (messageExchange.getSender() is IVisioExportableWithShape
exportableSender)
            shape.CellsU["BeginX"].GlueToPos(exportableSender.getShape(), 1, 0.5);

        if (messageExchange.getReceiver() is IVisioExportableWithShape
exportableReceiver)
            shape.CellsU["EndY"].GlueToPos(exportableReceiver.getShape(), 0, 0.5);

        shape.CellsU["User.globalX"].FormulaU = "\"\" +
(shape.CellsU["BeginX"].Result[""] + shape.CellsU["EndX"].Result[""])/2.0 + "\"";
        shape.CellsU["User.globalY"].FormulaU = "\"\" +
(shape.CellsU["BeginY"].Result[""] + shape.CellsU["EndY"].Result[""]) / 2.0 +
"\"";

```

This centers the message box on the connector.

```

        shape.CellsU["Actions.Row_1.Action"].Trigger();
    }
}

```

OWLShapes/InteractionDescribing/VisioMessageExchangeList.cs

```

public class VisioMessageExchangeList : MessageExchangeList,
IVisioExportableWithShape
{
    private const string type =
ALPSConstants.alpsSIDMasterStandardMessageConnector;

    private readonly IShapeExport export;
    protected VisioMessageExchangeList()
    {
        export = new MessageExchangeListExport(this);
    }
}

```

```
public void exportToVisio(Visio.Page currentPage)
{
```

For some reason a check exists, if this object's shape already got exported.

```
    if (getShape() != null) return;

    export.export(VisioHelper.ShapeType.SID, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
```

The [VisioMessageSpecifications](#) are exported to the current page.

```
    foreach (IMessageExchange messageExchange in getMessageExchanges().Values)
    {
        if (messageExchange.getMessageType() is IVisioExportable exportable)
        exportable.exportToVisio(currentPage);
    }
}
```

OWLShapes/ExportFunctionality/MessageExchangeListExport.cs

```
public class MessageExchangeListExport : PASSProcessModelElementExport
{
    readonly IMessageExchangeList messageExchangeList;
    public MessageExchangeListExport(IMessageExchangeList messageSpecification) :
base(messageSpecification)
    {
        this.messageExchangeList = messageSpecification;
    }

    public override void export(ShapeType shapeType, Visio.Page page, string
masterType, IList<ISimple2DVisualizationPoint> points = null,
IPASSProcessModelElement originalElement = null)
    {
        base.export(shapeType, page, masterType, points, originalElement);
    }
}
```

OWLShapes/InteractionDescribing/VisioMessageSpecification.cs

```

public class VisioMessageSpecification : MessageSpecification,
IVisioExportableWithShape
{
    private const string type = ALPSConstants.alpsSIDMasterMessage;

    private readonly IShapeExport export;
    protected VisioMessageSpecification()
    {
        export = new MessageSpecificationExport(this);
    }

    public void exportToVisio(Visio.Page currentPage)
    {

```

For some reason a check exists, if this object's shape already got exported.

```

        if (getShape() != null) return;

        export.export(VisioHelper.ShapeType.SID, currentPage, type, new
List<ISimple2DVisualizationPoint>
(getElementsWithUnspecifiedRelation().Values.OfType<ISimple2DVisualizationPoint>
()), this);
    }
}

```

OWLShapes/ExportFunctionality/MessageSpecificationExport.cs

```

public class MessageSpecificationExport : PASSProcessModelElementExport
{
    readonly IMessageSpecification messageSpecification;
    public MessageSpecificationExport(IMessageSpecification messageSpecification)
: base(messageSpecification)
    {
        this.messageSpecification = messageSpecification;
    }

    public override void export(ShapeType shapeType, Visio.Page page, string
masterType, IList<ISimple2DVisualizationPoint> points = null,
IPASSProcessModelElement originalElement = null)
    {
        base.export(shapeType, page, masterType, points, originalElement);
    }
}

```

Unknown Functions

The following classes are not documented here, because their purpose is unknown to the author. They may be implemented in a similar way to the other classes mentioned here.

- VisioMacroBehavior
- VisioSubjectBehavior
- VisioCommunicationRestriction
- VisioInterfaceSubject
- VisioStandAloneMacroSubject
- VisioGenericReturnToOriginReference
- VisioFlowRestrictor